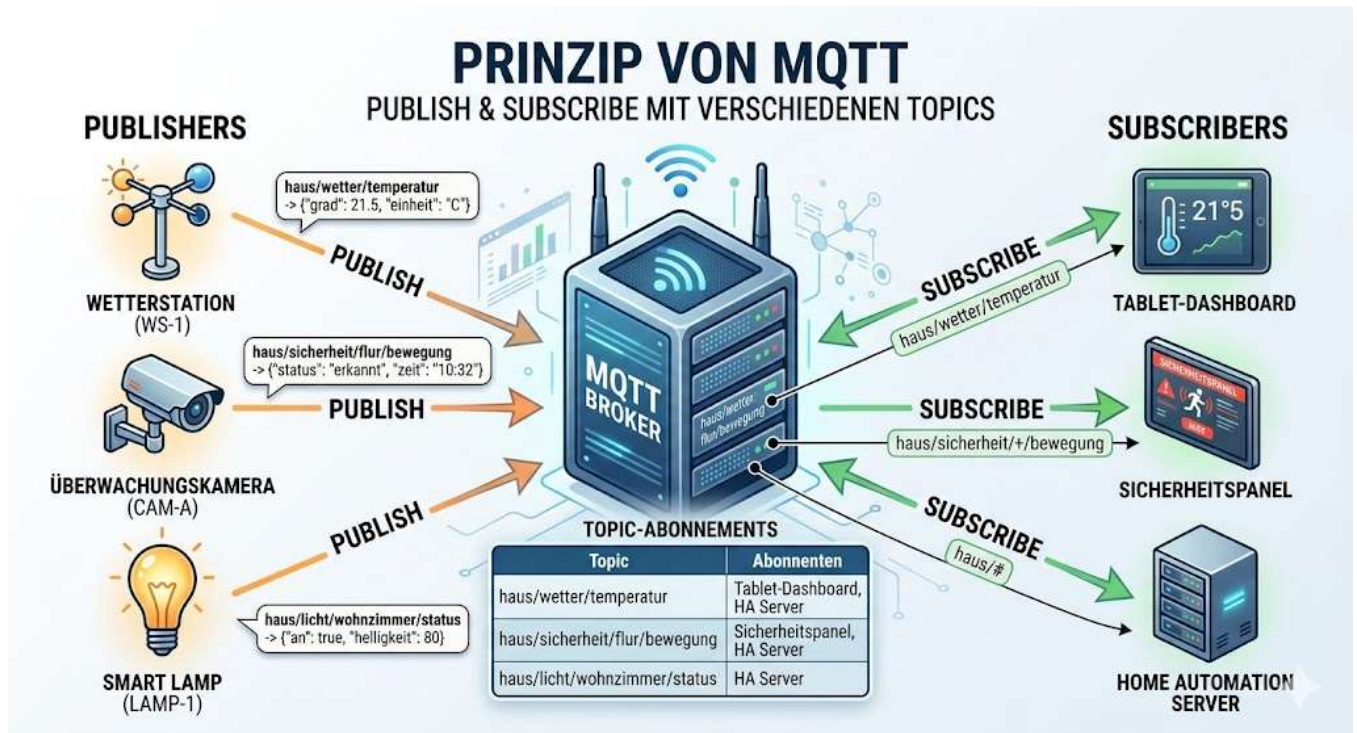


04 - AB: MQTT Funktionsweise

Mittwoch, 10. Juni 2026 16:31



MQTT (Message Queuing Telemetry Transport) ist ein leichtgewichtiges, offenes Kommunikationsprotokoll, das besonders für den Einsatz in IoT (Internet of Things)-Anwendungen entwickelt wurde. Es ermöglicht den Datenaustausch zwischen Geräten (Clients) über einen zentralen Server, den sogenannten **Broker**. Dabei basiert es auf einem **Publish/Subscribe-Modell**.

Begriffe:

- **Broker:** Der zentrale Server, der Nachrichten empfängt, speichert und an Abonnenten weiterleitet.
- **Client:** Ein Gerät oder eine Anwendung, die Nachrichten sendet (**publish**) oder empfängt (**subscribe**).
- **Topic:** Ein Thema oder Kanal, über den Nachrichten ausgetauscht werden.
- **Publish:** Das Senden einer Nachricht an ein bestimmtes Topic.
- **Subscribe:** Das Abonnieren eines Topics, um Nachrichten zu diesem Thema zu erhalten.

1. Funktionsweise der Subscription

- Ein **Client** (z. B. ein IoT-Gerät oder eine Anwendung) sendet eine **Subscribe-Nachricht** an den Broker.
- In dieser Nachricht gibt der Client das spezifische **Topic** oder einen Topic-Filter (z. B. `sensor/temperatur/#`) an, auf das er Nachrichten erhalten möchte.
- Der **Broker** registriert die Anfrage und merkt sich, dass dieser Client Nachrichten für das angegebene Topic abonnieren möchte.

2. Verantwortlichkeiten

- **Subscriber (abonnierender Client):**
 - Sendet eine **Subscribe-Nachricht** mit dem gewünschten Topic an den Broker.
 - **Empfängt vom Broker Nachrichten**, die zu dem abonnierten Topic gehören.
 - Kann eine Quality of Service (QoS)-Stufe angeben, die die Zuverlässigkeit der Zustellung bestimmt (z. B. QoS 0, 1 oder 2).
- **Broker:**
 - Verarbeitet die **Subscribe-Anfragen** und speichert die Informationen zu den Abonnements (z. B. welcher Client welches Topic abonniert hat).
 - Leitet Nachrichten, die an ein Topic veröffentlicht werden, sofort an *alle* abonnierten Clients weiter. Dazu müssen diese online sein und eine aktive TCP-Verbindung zum Broker aufgebaut haben. Wenn QoS 1 oder 2 angegeben, werden die Nachrichten zwischengespeichert.
- **Publisher (nachrichtensendender Client):**
 - Veröffentlicht Nachrichten für ein bestimmtes Topic, ohne zu wissen, wer sie empfängt.

3. Speicherung von Informationen

- Der **Broker** speichert:
 - Eine Liste aller **Topics** und der zugehörigen **abonnierten Clients**.
 - Nachrichten mit gesetztem **Retain-Flag**, um neuen Abonnenten später die jeweils aktuellste Nachricht direkt bereitzustellen.
 - Nachrichten mit gesetztem **Last Will-Flag**, um bei Ausfall des Publishers diese spezielle Benachrichtigung versenden zu können.
 - **Temporäre Nachrichten in einer Warteschlange**, falls ein abonnierender Client gerade nicht erreichbar ist (bei **QoS 1 oder 2**).

Der Subscriber selbst speichert typischerweise keine Informationen über Topics, da der Broker diese Logik übernimmt.

Wie lange bleiben die Nachrichten auf dem MQTT-Broker gespeichert?

Die Speicherdauer von Nachrichten auf einem MQTT-Broker hängt von der sogenannten **QoS-Stufe** (Quality of Service) und dem **Retain-Flag** der Nachricht ab. Es gibt drei Fälle:

1. **Nachrichten ohne Retain-Flag:**
Diese Nachrichten werden direkt an abonnierte Clients weitergeleitet und nicht auf dem Broker gespeichert.
2. **Nachrichten mit Retain-Flag:**
Wenn das **Retain-Flag** aktiviert ist, speichert der Broker die Nachricht für das angegebene Topic, bis sie durch eine neue retained Nachricht ersetzt wird. Dies ist nützlich, um neuen Abonnenten direkt die aktuellste Nachricht zu liefern.
3. **Nachrichten in einer Warteschlange (QoS 1 oder 2):**
Wenn ein Client mit QoS 1 oder 2 abonniert, der momentan nicht verfügbar ist, speichert der

Broker die Nachrichten temporär in einer Warteschlange, bis der Client sie abrufen kann. Die Dauer hängt hier davon ab, wie der Broker konfiguriert ist (z. B. Speicherzeit oder maximale Nachrichtenanzahl).

Die genaue Speicherdauer hängt also von den Einstellungen des Brokers (z. B. Mosquitto oder HiveMQ) und den Konfigurationen der Nachricht ab.